



UNIVERSIDAD DE LAS FUERZAS ARMADAS
ESPE

INGENIERÍA DE SOFTWARE

Análisis y Diseño de Software
30724

**Relación entre casos de uso, diagrama de clases de
análisis y diagramas de secuencia de análisis**

Presenta:

Cuzco Yamasca Alex Darío

Domínguez Aguilera Pablo Alejandro

Alvarado Palacios Dylan Alexander

Docente:

Jenny Alexandra Ruiz Robalino

Fecha:

05 de Mayo de 2026



Índice

1. Objetivos	3
1.1. Objetivo general	3
1.2. Objetivos específicos	3
2. Desarrollo	3
2.1. Requisitos funcionales Nivel 0	4
2.2. Diagrama de casos de uso de análisis	4
2.3. Derivación de clases de análisis	8
2.4. Matriz de relación: casos de uso, clases y secuencias	13
2.5. Diagramas de secuencia de análisis	14
2.5.1. CU-001: Acceso al Software	14
2.5.2. CU-003: Consultar Calendario de Eventos	16
2.5.3. CU-004: Proponer Evento	19
2.5.4. CU-005: Confirmar Asistencia a Evento	22
2.6. Especificación breve de casos de uso conectada con las secuencias	25
3. Conclusiones	26
4. Recomendaciones	27

1. Objetivos

1.1. Objetivo general

Analizar los diagramas de casos de uso, el diagrama de clases de análisis y los diagramas de secuencia para implementar el sistema EVENTUAL orientado a la gestión de eventos del Club de Suboficiales, mediante el modelado en PlantUML, garantizando la trazabilidad integral entre los requisitos funcionales de nivel 0 y la interacción dinámica de los objetos en procesos como la autenticación de usuarios, la consulta de eventos, la propuesta de actividades y la confirmación de asistencia.

1.2. Objetivos específicos

- Modelar el comportamiento funcional del sistema EVENTUAL mediante diagramas de casos de uso, identificando las interacciones de los actores (Socio, Presidente, Tesorero y Secretario) con funcionalidades como autenticación, consulta de eventos, propuesta de actividades y confirmación de asistencia.
- Construir el diagrama de clases de análisis utilizando los estereotipos boundary, control, entity y repository, con el fin de definir la estructura del sistema y garantizar la coherencia entre los requisitos funcionales y los componentes del modelo.
- Validar la lógica del sistema a través de diagramas de secuencia de análisis, detallando el flujo de mensajes entre objetos en cada caso de uso, asegurando que los procesos definidos respondan de manera correcta a los eventos generados por los actores.

2. Desarrollo

Para el desarrollo del presente análisis se ha tomado como base el sistema EVENTUAL, orientado a la gestión de eventos del Club de Suboficiales. El proceso se estructura de manera progresiva, iniciando con la identificación de los requisitos funcionales de nivel 0 y avanzando hacia la representación del comportamiento y la estructura del sistema mediante diagramas de casos de uso, clases de análisis y diagramas de secuencia. Este enfoque permite establecer una relación clara entre las necesidades de los actores (Socio, Presidente, Tesorero y Secretario) y las soluciones propuestas, asegurando la trazabilidad entre los requerimientos y la interacción dinámica de los objetos. De esta forma, se garantiza un modelo coherente que servirá como base para las etapas posteriores de diseño e implementación del sistema.

2.1. Requisitos funcionales Nivel 0

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-01	El software debe permitir la autenticación de usuarios mediante cédula y contraseña.	Socio, Tesorero, Presidente, Secretario	Acceso al software
RF-03	El sistema debe mostrar una vista mensual de eventos programados con información detallada.	Socio, Tesorero, Presidente, Secretario	Consultar calendario de eventos
RF-04	Los socios deben poder proponer eventos (sociales o deportivos) mediante un formulario digital.	Socio	Proponer evento

2.2. Diagrama de casos de uso de análisis

El diagrama de casos de uso representa las interacciones entre los diferentes actores del sistema EVENTUAL (Socio, Presidente, Tesorero y Secretario) y las funcionalidades principales del software relacionadas con la gestión de eventos. Este modelo permite visualizar el alcance del sistema, identificando las responsabilidades de cada actor y describiendo cómo se llevarán a cabo procesos clave como el acceso al software, la consulta del calendario y la propuesta de eventos. Además, se incorporan relaciones como <<include>> y <<extend>> para reflejar comportamientos obligatorios y opcionales, garantizando una representación clara, estructurada y coherente con los requisitos definidos en el SRS.

```
@startuml Sistema de Planificación de Eventos
```

```
left to right direction
skinparam defaultFontName Arial
skinparam defaultFontSize 11
skinparam backgroundColor white
skinparam ActorBorderColor #333333
skinparam ActorBackgroundColor white
skinparam UseCaseBorderColor #555555
skinparam UseCaseBackgroundColor #EEEEEE
skinparam ArrowColor #333333
skinparam NoteBackgroundColor #FFFC0
skinparam NoteBorderColor #999999
skinparam NoteFontSize 10
skinparam PackageBorderColor #333333
```

```

skinparam PackageBackgroundColor white
skinparam PackageFontStyle bold

title Sistema de Planificación de Eventos

actor "Usuario" as usuario
actor "Socio" as socio
actor "Tesorero" as tesorero
actor "Presidente" as presidente

rectangle "Casos de Uso Principales" as CUP {
    usecase "CU-001\nAcceso al Software" as CU001
    usecase "CU-003\nConsultar Calendario\nde Eventos" as CU003
    usecase "CU-004\nProponer Evento" as CU004
}

rectangle "Procesos Internos" as PI {
    usecase "Validar credenciales" as VC
    usecase "Generar sesión" as GS
    usecase "Filtrar eventos" as FE
    usecase "Validar propuesta" as VP
    usecase "Notificar directiva" as ND
}

note right of VC
    **Ingreso con:**
    * Cédula
    * Contraseña
end note

note right of GS
    **Permite visualizar:**
    * Eventos mensuales
    * Tipo de evento
    * Detalle del evento
end note

note right of VP
    **Datos de propuesta:**
    * Tipo
    * Descripción
    * Fecha sugerida
    * Justificación
end note

' Relaciones actores izquierda
usuario --> CU001
usuario --> CU003

```

```
socio --> CU004
```

```
' Include / Extend
```

```
CU001 ..> VC : <<include>>
```

```
CU001 ..> GS : <<include>>
```

```
CU003 ..> FE : <<extend>>
```

```
CU004 ..> VP : <<include>>
```

```
CU004 ..> ND : <<include>>
```

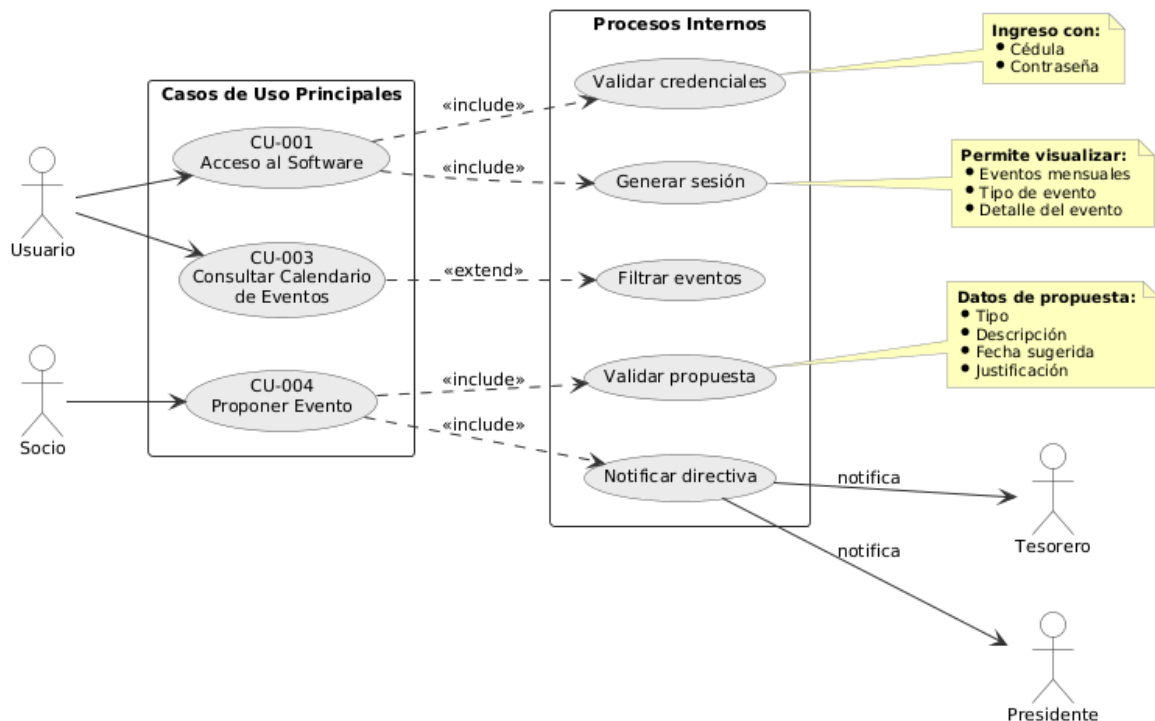
```
' Notificaciones actores derecha
```

```
ND --> tesorero : notifica
```

```
ND --> presidente : notifica
```

```
@enduml
```

Sistema de Planificación de Eventos



2.3. Derivación de clases de análisis

A partir de los casos de uso se identifican clases de análisis. Para mantener una separación clara de responsabilidades se recomienda usar los estereotipos boundary, control y entity. Además, se puede incluir un repositorio para representar la persistencia o colección de estudiantes sin entrar aún en detalles técnicos de base de datos.

Tipo de clase	Responsabilidad en el análisis	Clase propuesta	Relación con los Casos de Uso
Boundary / Frontera	Recibe credenciales del actor y muestra resultado de autenticación.	PantallaLogin	CU-001: pantalla con campos Cédula, Contraseña y botón Iniciar Sesión.
Boundary / Frontera	Muestra la vista mensual del calendario con eventos y controles de filtro/recordatorio.	PantallaCalendario	CU-003: presenta eventos por mes, filtros por tipo y opción de recordatorio.
Boundary / Frontera	Muestra el detalle de un evento seleccionado.	PantallaDetalleEvento	CU-003: despliega nombre, fecha, lugar y aforo
Boundary / Frontera	Recibe los datos de la propuesta del Socio y muestra el número de seguimiento.	FormularioPropuestaEvento	CU-004: campos Tipo, Descripción, Fecha Sugerida, Justificación y botón Enviar.
Control	Coordina la autenticación: valida credenciales, gestiona intentos fallidos, genera token.	ControlAcceso	CU-001: lógica de validación, bloqueo de cuenta y creación de sesión.
Control	Coordina la consulta del calendario: carga eventos, aplica filtros y programa recordatorios.	ControlCalendario	CU-003: lógica de carga mensual, filtrado por tipo y configuración de notificaciones.
Control	Coordina el registro de propuestas: valida formulario, genera ID y dispara notificaciones.	ControlPropuestaEvento	CU-004: valida datos, genera PRO-AAAA-NNN, notifica a Tesorero y Presidente.
Entity / Entidad	Representa la información de un usuario del sistema con su rol y estado.	Usuario	Todos los CU: contiene cédula, nombre, contraseña, rol y estado (Activo/Inactivo).
Entity / Entidad	Representa la información de un evento planificado en el club.	Evento	CU-003 / CU-004 : contiene nombre, fecha, lugar, tipo, aforo y estado.

Entity / Entidad	Representa una propuesta de evento enviada por un Socio.	PropuestaEvento	CU-004: contiene ID (PRO-AAAA-NNN), descripción, fecha sugerida, justificación y estado.
Entity / Entidad	Representa la sesión activa de un usuario autenticado.	SesionUsuario	CU-001: contiene token, timestamp de inicio y expiración (60 min inactividad).
Repositorio / Colección	Conserva y recupera registros de usuarios del sistema.	RepositorioUsuario	CU-001: busca por cédula, verifica estado activo, guarda log de accesos.
Repositorio / Colección	Conserva y recupera eventos planificados del club.	RepositorioEvento	CU-003: lista por mes, filtra por tipo, actualiza plazas confirmadas.
Repositorio / Colección	Conserva y recupera propuestas de eventos enviadas por socios.	RepositorioPropuestaEvento	CU-004: guarda propuestas, asigna ID correlativo, permite consulta de seguimiento.

```
@startuml
```

```
left to right direction
```

```
skinparam defaultFontName Arial
skinparam defaultFontSize 11
skinparam backgroundColor white
skinparam ClassBorderColor #333333
skinparam ClassBackgroundColor #FFFFFF
skinparam ClassHeaderBackgroundColor #EEEEEE
skinparam ArrowColor #333333
skinparam ClassFontStyle bold
```

```
class PantallaLogin <<Boundary>> {
    - campoCedula : String
    - campoContrasena : String
    - botonIniciarSesion : Button
    --
    + mostrarFormulario() : void
}
```



```

+ capturarCredenciales() : Credenciales
+ mostrarErrorAutenticacion(mensaje : String) : void
+ mostrarSesionExitosa() : void
}

class PantallaCalendario <<Boundary>> {
- vistaActual : String
- mesSeleccionado : Date
- filtroPorTipo : String
--
+ mostrarCalendarioMensual(mes : Date) : void
+ aplicarFiltro(tipo : String) : void
+ mostrarEventos(eventos : List) : void
+ configurarRecordatorio(evento : Evento) : void
+ seleccionarEvento(eventoId : String) : void
}

class PantallaDetalleEvento <<Boundary>> {
- nombreEvento : String
- fechaEvento : Date
- lugarEvento : String
- aforoEvento : Integer
--
+ mostrarDetalle(evento : Evento) : void
+ cerrarDetalle() : void
}

class FormularioPropuestaEvento <<Boundary>> {
- campoTipo : String
- campoDescripcion : String
- campoFechaSugerida : Date
- campoJustificacion : String
- botonEnviar : Button
- numeroSeguimiento : String
--
+ mostrarFormulario() : void
+ capturarDatosPropuesta() : PropuestaEvento
+ validarCampos() : Boolean
+ mostrarNumeroSeguimiento(id : String) : void
+ enviarPropuesta() : void
}

```

```
' =====  
' CONTROL  
' =====
```

```
class ControlAcceso <<Control>> {  
    - intentosFallidos : Integer  
    - tokenSesion : String  
    --  
    + validarCredenciales(cedula : String, contrasena : String) : Boolean  
    + gestionarIntentosFallidos(cedula : String) : void  
    + bloquearCuenta(cedula : String) : void  
    + generarToken() : String  
    + cerrarSesion(token : String) : void  
}
```

```
class ControlCalendario <<Control>> {  
    - mesActual : Date  
    - tipoFiltro : String  
    --  
    + cargarEventosMensuales(mes : Date) : List  
    + filtrarPorTipo(tipo : String, eventos : List) : List  
    + programarRecordatorio(evento : Evento) : void  
    + configurarNotificacion(usuarioId : String, eventoId : String) : void  
}
```

```
class ControlPropuestaEvento <<Control>> {  
    - idCorrelativo : String  
    --  
    + validarFormulario(propuesta : PropuestaEvento) : Boolean  
    + generarId() : String  
    + registrarPropuesta(propuesta : PropuestaEvento) : String  
    + notificarTesorero(propuesta : PropuestaEvento) : void  
    + notificarPresidente(propuesta : PropuestaEvento) : void  
}
```

```
' =====  
' ENTITY / ENTIDAD  
' =====
```

```
class Usuario <<Entity>> {
```

```
- cedula : String
- nombre : String
- contrasena : String
- rol : String
- estado : String
--
+ getCedula() : String
+ getNombre() : String
+ getRol() : String
+ getEstado() : String
+ isActive() : Boolean
}

class Evento <<Entity>> {
  - nombre : String
  - fecha : Date
  - lugar : String
  - tipo : String
  - aforo : Integer
  - estado : String
  --
  + getNombre() : String
  + getFecha() : Date
  + getLugar() : String
  + getTipo() : String
  + getAforo() : Integer
  + getEstado() : String
}

class PropuestaEvento <<Entity>> {
  - id : String
  - descripcion : String
  - fechaSugerida : Date
  - justificacion : String
  - estado : String
  --
  + getId() : String
  + getDescripcion() : String
  + getFechaSugerida() : Date
  + getJustificacion() : String
  + getEstado() : String
}
```

```

    + setEstado(estado : String) : void
}

class SesionUsuario <<Entity>> {
    - token : String
    - timestampInicio : DateTime
    - timestampExpiracion : DateTime
    - tiempoInactividad : Integer
    --
    + getToken() : String
    + getTimestampInicio() : DateTime
    + getTimestampExpiracion() : DateTime
    + isVigente() : Boolean
    + renovarSesion() : void
    + cerrarSesion() : void
}

' =====
' REPOSITORIO / COLECCIÓN
' =====

class RepositorioUsuario <<Repository>> {
    --
    + buscarPorCedula(cedula : String) : Usuario
    + verificarEstadoActivo(cedula : String) : Boolean
    + guardarLogAcceso(cedula : String, fecha : DateTime) : void
    + actualizarEstado(cedula : String, estado : String) : void
}

class RepositorioEvento <<Repository>> {
    --
    + listarPorMes(mes : Date) : List
    + filtrarPorTipo(tipo : String) : List
    + actualizarPlazasConfirmadas(eventoId : String, plazas : Integer) : void
    + buscarPorId(eventoId : String) : Evento
}

class RepositorioPropuestaEvento <<Repository>> {
    --
    + guardarPropuesta(propuesta : PropuestaEvento) : void
    + asignarIdCorrelativo(propuesta : PropuestaEvento) : String
}

```

```

+ buscarPorId(id : String) : PropuestaEvento
+ consultarSeguimiento(id : String) : PropuestaEvento
+ listarPorEstado(estado : String) : List
}

' =====
' RELACIONES
' =====

' Boundary -> Control
PantallaLogin ..> ControlAcceso : usa
PantallaCalendario ..> ControlCalendario : usa
PantallaDetalleEvento ..> ControlCalendario : usa
FormularioPropuestaEvento ..> ControlPropuestaEvento : usa

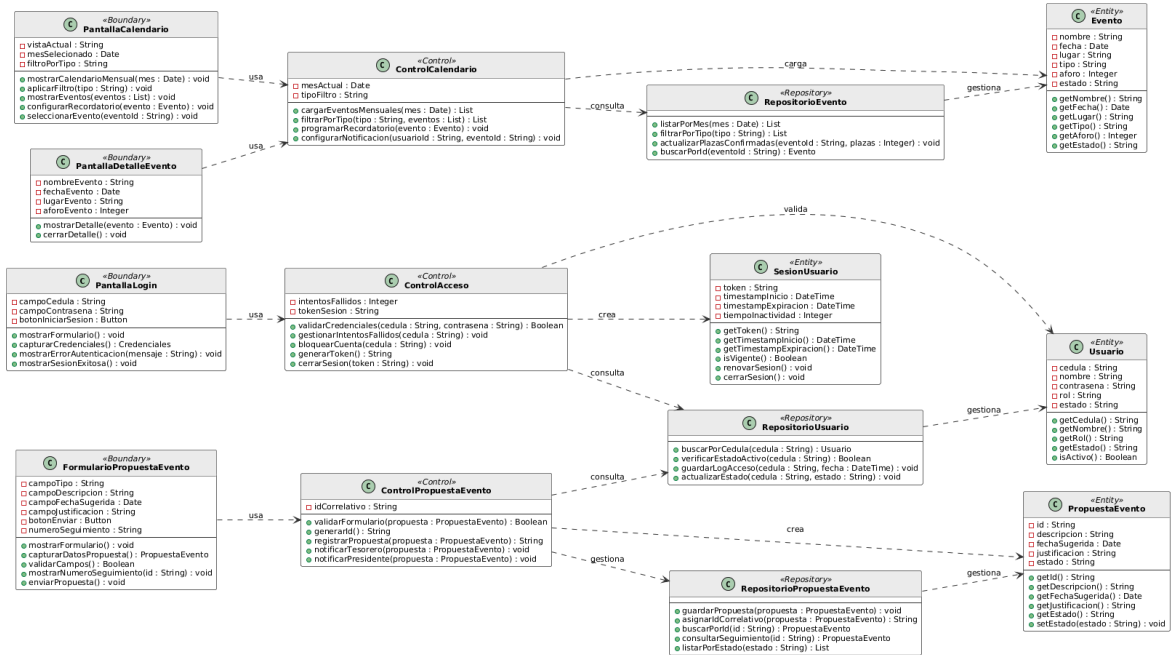
' Control -> Repositorio
ControlAcceso ..> RepositorioUsuario : consulta
ControlCalendario ..> RepositorioEvento : consulta
ControlPropuestaEvento ..> RepositorioPropuestaEvento : gestiona
ControlPropuestaEvento ..> RepositorioUsuario : consulta

' Control -> Entity
ControlAcceso ..> SesionUsuario : crea
ControlAcceso ..> Usuario : valida
ControlCalendario ..> Evento : carga
ControlPropuestaEvento ..> PropuestaEvento : crea

' Repositorio -> Entity
RepositorioUsuario ..> Usuario : gestiona
RepositorioEvento ..> Evento : gestiona
RepositorioPropuestaEvento ..> PropuestaEvento : gestiona

@enduml

```



2.4. Matriz de relación: casos de uso, clases y secuencias

Caso de uso	Boundary	Control	Entity / Repositorio	Diagrama de secuencia esperado
CU-001: Acceso al Software	PantallaLogin captura cédula y contraseña, muestra mensajes de error o acceso.	ControlAcceso valida campos, verifica credenciales, controla intentos fallidos y genera sesión.	Consulta Usuario en RepositorioUsuario, crea SesionUsuario y registra acceso.	Ingresar credenciales → validar campos → verificar credenciales → controlar intentos → generar sesión → redirigir según rol.
CU-003: Consultar Calendario de Eventos	PantallaCalen dario muestra eventos y filtros. PantallaDetal leEvento muestra	ControlCalendar io carga eventos, aplica filtros y obtiene detalles.	Consulta Evento en RepositorioEvento y obtiene información detallada.	Seleccionar calendario → cargar eventos → mostrar lista → aplicar filtro (opcional) → seleccionar

	información del evento.			evento → mostrar detalle.
CU-004: Proponer Evento	FormularioPropuestaEvento captura tipo, descripción, fecha y justificación.	ControlPropuestaEvento valida datos, genera ID y gestiona registro.	Crea PropuestaEvento en RepositorioPropuestaEvento y consulta usuarios en RepositorioUsuario para notificación.	Acceder a formulario → ingresar datos → validar → generar ID → registrar propuesta → notificar → mostrar confirmación.

2.5. Diagramas de secuencia de análisis

2.5.1. CU-001: Acceso al Software

@startuml

```
skinparam defaultFontName Arial
skinparam defaultFontSize 11
skinparam backgroundColor skyblue
skinparam SequenceArrowThickness 1.2
skinparam ParticipantBorderColor #333333
skinparam ParticipantBackgroundColor #EEEEEE
skinparam ActivationBorderColor #333333
skinparam ActivationBackgroundColor #DDDDDD
```

title CU-001: Acceso al Software

```
actor Usuario as usr
participant "PantallaLogin" as PL <<Boundary>>
participant "ControlAcceso" as CA <<Control>>
participant "RepositorioUsuario" as RU <<Repository>>
participant "Usuario" as U <<Entity>>
participant "SesionUsuario" as SU <<Entity>>
```

```
usr -> PL : ingresa credenciales\n(cedula, contrasena)
activate PL
```

```

PL -> PL : capturarCredenciales()
PL -> CA : validarCredenciales(cedula, contrasena)
activate CA

CA -> RU : buscarPorCedula(cedula)
activate RU
RU --> CA : Usuario
deactivate RU

CA -> U : isActive()
activate U
U --> CA : Boolean
deactivate U

alt credenciales correctas y cuenta activa
    CA -> CA : generarToken()
    CA -> SU : <<create>>\ntoken, timestampInicio,\ntimestampExpiracion
    activate SU
    SU --> CA : SesionUsuario
    deactivate SU
    CA -> RU : guardarLogAcceso(cedula, timestampInicio)
    activate RU
    RU --> CA : void
    deactivate RU
    CA --> PL : sesion valida + rol
    PL -> PL : mostrarSesionExitosa()
    PL --> usr : redirige según rol

else credenciales incorrectas
    CA -> CA : gestionarIntentosFallidos(cedula)
    alt intentosFallidos >= limite
        CA -> CA : bloquearCuenta(cedula)
        CA -> RU : actualizarEstado(cedula, "Inactivo")
        activate RU
        RU --> CA : void
        deactivate RU
        CA --> PL : cuenta bloqueada
        PL -> PL : mostrarErrorAutenticacion("Cuenta bloqueada")
        PL --> usr : muestra error de bloqueo
    else intentos aún permitidos
        CA --> PL : credenciales invalidas

```



```

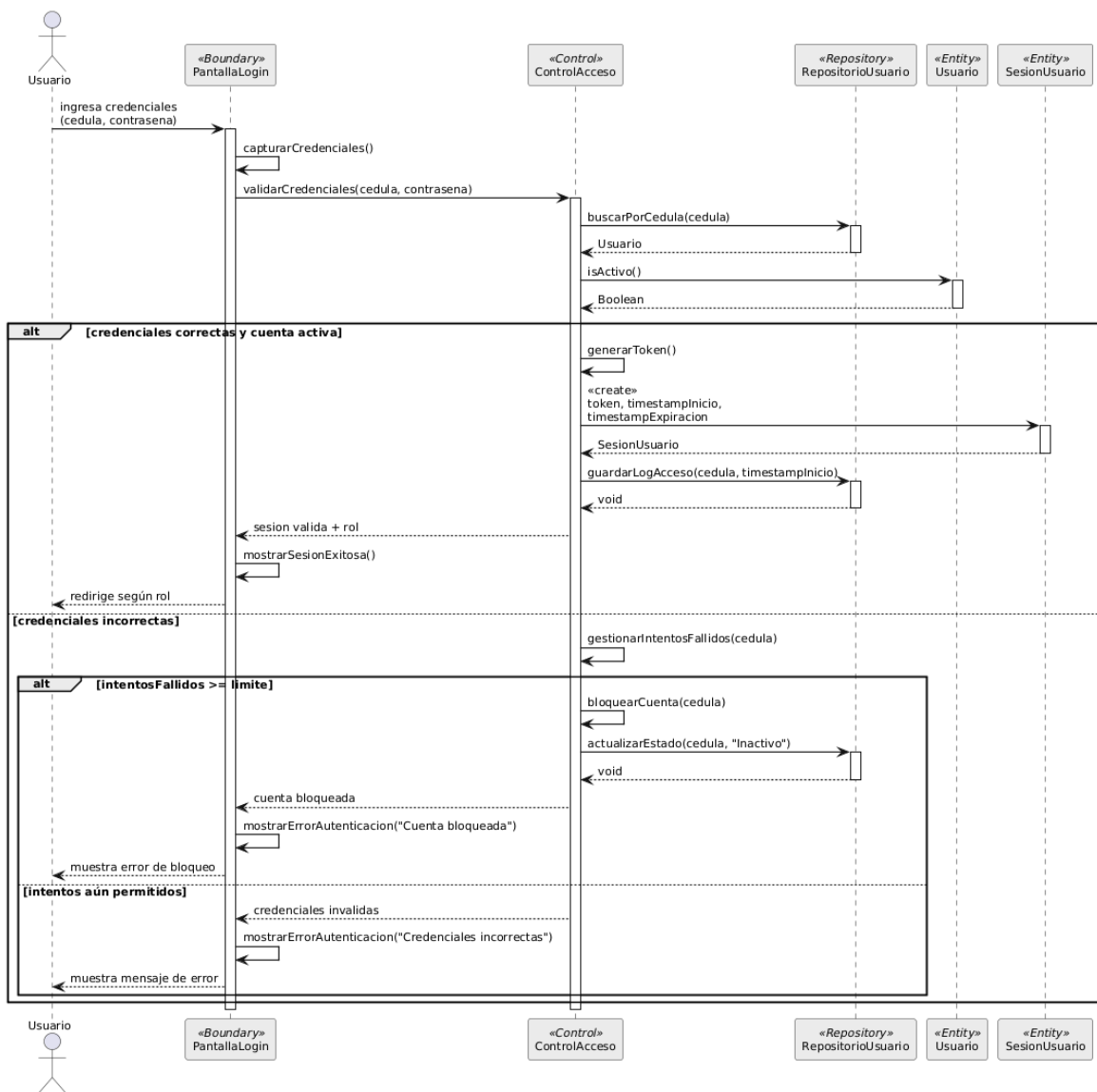
        PL -> PL : mostrarErrorAutenticacion("Credenciales incorrectas")
        PL --> usr : muestra mensaje de error
    end
end

deactivate CA
deactivate PL

@enduml

```

CU-001: Acceso al Software



2.5.2. CU-003: Consultar Calendario de Eventos

```
@startuml
CU-003 Consultar Calendario de Eventos

skinparam defaultFontName Arial
skinparam defaultFontSize 11
skinparam backgroundColor white
skinparam SequenceArrowThickness 1.2
skinparam ParticipantBorderColor #333333
skinparam ParticipantBackgroundColor #EEEEEE

title CU-003: Consultar Calendario de Eventos

actor Usuario as usr
participant "PantallaCalendario" as PC <<Boundary>>
participant "PantallaDetalleEvento" as PDE <<Boundary>>
participant "ControlCalendario" as CC <<Control>>
participant "RepositorioEvento" as RE <<Repository>>

usr -> PC : selecciona mes
PC -> CC : cargarEventosMensuales(mesSeleccionado)
CC -> RE : listarPorMes(mes)
RE --> CC : List<Evento>
CC --> PC : List<Evento>
PC --> usr : muestra calendario con eventos

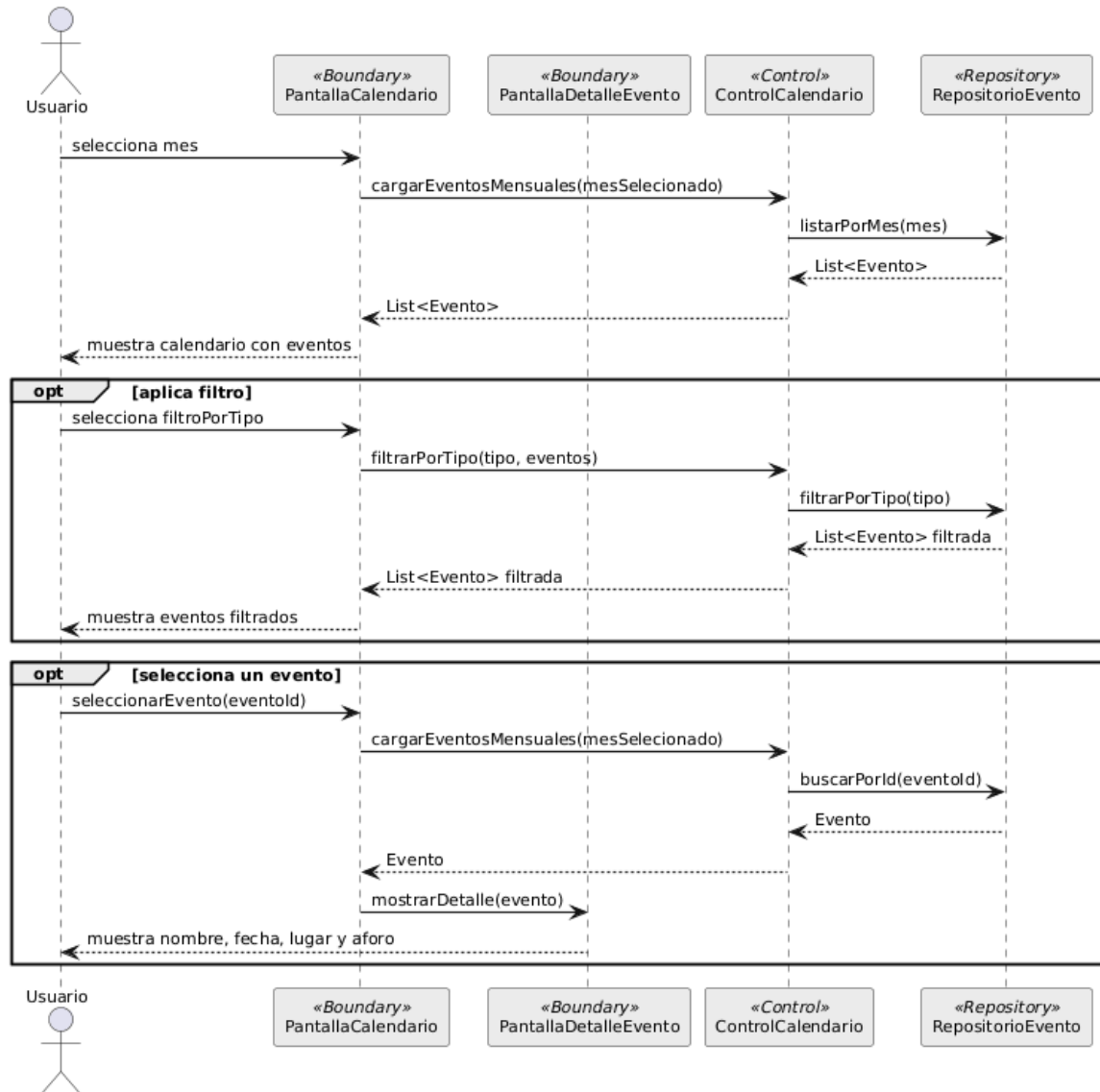
opt aplica filtro
    usr -> PC : selecciona filtroPorTipo
    PC -> CC : filtrarPorTipo(tipo, eventos)
    CC -> RE : filtrarPorTipo(tipo)
    RE --> CC : List<Evento> filtrada
    CC --> PC : List<Evento> filtrada
    PC --> usr : muestra eventos filtrados
end

opt selecciona un evento
    usr -> PC : seleccionarEvento(eventoId)
    PC -> CC : cargarEventosMensuales(mesSeleccionado)
    CC -> RE : buscarPorId(eventoId)
    RE --> CC : Evento
    CC --> PC : Evento
    PC -> PDE : mostrarDetalle(evento)
```

```
PDE --> usr : muestra nombre, fecha, lugar y aforo
end

@enduml
```

CU-003: Consultar Calendario de Eventos



2.5.3. CU-004: Proponer Evento

```
@startuml

skinparam defaultFontName Arial
skinparam defaultFontSize 11
skinparam backgroundColor white
skinparam SequenceArrowThickness 1.2
skinparam ParticipantBorderColor #333333
skinparam ParticipantBackgroundColor #EEEEEE

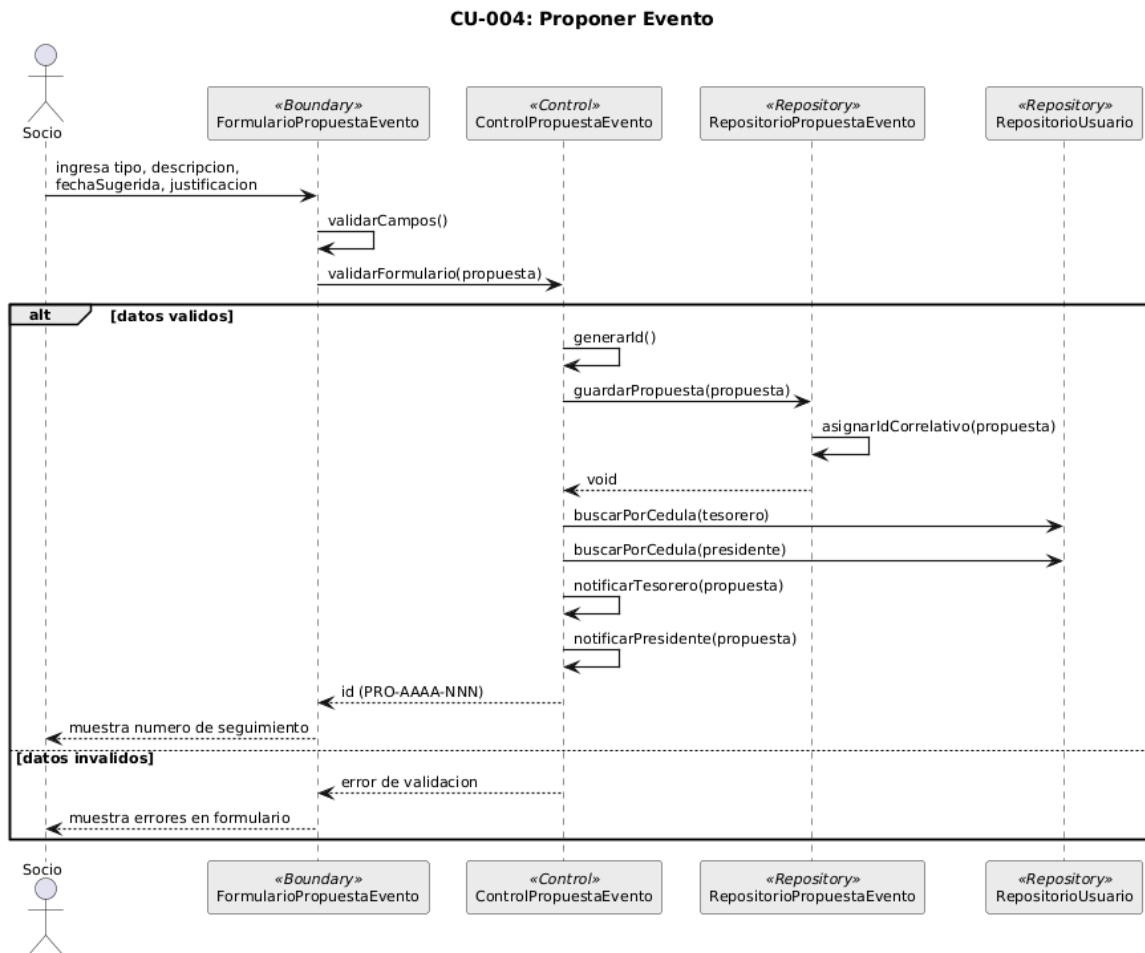
title CU-004: Proponer Evento

actor Socio as socio
participant "FormularioPropuestaEvento" as FPE <<Boundary>>
participant "ControlPropuestaEvento" as CPE <<Control>>
participant "RepositorioPropuestaEvento" as RPE <<Repository>>
participant "RepositorioUsuario" as RU <<Repository>>

socio -> FPE : ingresa tipo, descripcion,\nfechaSugerida, justificacion
FPE -> FPE : validarCampos()
FPE -> CPE : validarFormulario(propuesta)

alt datos validos
    CPE -> CPE : generarId()
    CPE -> RPE : guardarPropuesta(propuesta)
    RPE -> RPE : asignarIdCorrelativo(propuesta)
    RPE --> CPE : void
    CPE -> RU : buscarPorCedula(tesorero)
    CPE -> RU : buscarPorCedula(presidente)
    CPE -> CPE : notificarTesorero(propuesta)
    CPE -> CPE : notificarPresidente(propuesta)
    CPE --> FPE : id (PRO-AAAA-NNN)
    FPE --> socio : muestra numero de seguimiento
else datos invalidos
    CPE --> FPE : error de validacion
    FPE --> socio : muestra errores en formulario
end

@enduml
```



2.6. Especificación breve de casos de uso conectada con las secuencias

Caso de uso	Precondición	Flujo principal resumido	Resultado esperado
CU-001: Acceso al Software	El usuario debe estar registrado y tener la cuenta activa	Ingresar cédula y contraseña → validar campos → verificar credenciales → controlar intentos fallidos → generar sesión → redirigir según rol	Usuario autenticado accede al sistema con sesión activa
CU-003: Consultar Calendario de Eventos	El usuario debe estar autenticado	Seleccionar opción calendario → cargar eventos del mes → mostrar vista → aplicar filtros (opcional) → seleccionar	Eventos visualizados correctamente y detalle accesible

		evento → mostrar detalle → configurar recordatorio (opcional)	
CU-004: Proponer Evento	El usuario (Socio) debe estar autenticado	Acceder a formulario → ingresar datos → validar campos → verificar descripción y fecha → generar ID de propuesta → registrar propuesta → notificar a directiva → mostrar confirmación	Propuesta registrada con identificador único y estado "Pendiente"

3. Conclusiones

- El modelado a partir de los requisitos funcionales de nivel 0 permitió estructurar de manera clara los casos de uso del sistema EVENTUAL, asegurando que procesos como la autenticación, consulta de eventos, propuesta de actividades y confirmación de asistencia respondan directamente a las necesidades de los actores.
- La integración de diagramas de casos de uso, clases de análisis y diagramas de secuencia permitió validar tanto la estructura como el comportamiento del sistema, garantizando coherencia entre la interacción del usuario y la lógica interna mediante el uso de estereotipos boundary, control, entity y repository.
- La trazabilidad entre requisitos, casos de uso, clases y secuencias asegura una transición ordenada hacia el diseño e implementación, reduciendo inconsistencias y facilitando la comprensión del flujo completo del sistema desde una perspectiva estática y dinámica.

4. Recomendaciones

- Es recomendable mantener consistencia entre los diagramas de clases y los diagramas de secuencia, verificando que todos los mensajes y operaciones estén correctamente definidos y correspondan a las responsabilidades de cada clase.
- También sería de gran ayuda mantener actualizada la matriz de trazabilidad cada vez que se realicen cambios en los requisitos o en el diseño, garantizando la coherencia entre todos los artefactos del análisis.

- Revisar detalladamente las reglas de negocio y que tengan relación directa con los diagramas de secuencia implementados (como validación de cupos, fechas y estados), ya que errores en esta etapa pueden afectar directamente la lógica del sistema en la implementación.